

SMART_AO V8 — ARC-01

Contrat d’ arborescence, modules et dépendances

Version : 1.0

Statut : contrat d’ architecture à appliquer lors de la création du dépôt V8

Auteur : Manus AI

Périmètre : dépôt monolithique modulaire, frontières de modules, règles d’ import, tests d’ architecture et plan de création incrémental

Dépendances normatives : Charte V8, DOMAIN-01 v1.1, DOMAIN-03 v1.0, APP-01 v1.0, TEST-01 v1.0 et DATA-01 v1.0

1. Objet et décision de gel

L’ arborescence n’ est pas un détail esthétique. Elle matérialise les frontières de changement : une règle métier, une commande, un modèle SQLAlchemy, une migration, un événement ou une projection doit pouvoir être situé immédiatement et ne pas contaminer le reste du logiciel.

Décision ARC-01 : SMART_AO V8 est un **monolithe modulaire**. Chaque module métier possède ses internals, son contrat public et ses tests. Un module voisin ne peut pas importer ses internals, sa session SQLAlchemy ou ses modèles ORM ; il passe par un contrat public, un événement ou une commande aval idempotente.

L’ arborescence cible décrite dans la proposition reçue est excellente comme **vision de maturité**. Elle serait toutefois dangereuse si elle créait dès le premier jour des dizaines de dossiers vides, des abstractions prématurées et des modules sans aggregate réel. ARC-01 sépare donc explicitement :

Niveau	Finalité	Règle
Arborescence de démarrage	Ce qui existe au premier commit V8.	Créer uniquement les modules et couches utiles au premier slice.
Arborescence cible	Ce que le dépôt pourra accueillir sans refonte.	Garder la place et les règles, sans créer de dossiers vides.

Création par slice	Moment où un futur module devient réel.	Un module apparaît lorsqu' il a un aggregate, une commande, un test et un besoin métier identifiés.
--------------------	---	---

2. Principes directeurs

ID	Principe	Règle pratique
ARC-01-01	Métier avant technique	Les dossiers principaux sont <code>case</code> , <code>dce</code> , <code>decision</code> , <code>pricing</code> , etc. ; jamais <code>models</code> , <code>schemas</code> , <code>engines</code> ou <code>agents</code> comme axe global.
ARC-01-02	Un root propriétaire	Un module ne sauvegarde que les aggregates qu' il possède selon DOMAIN-01.
ARC-01-03	Contrat public obligatoire	Un voisin importe seulement <code>module.public</code> , jamais <code>module.domain</code> ou <code>module.infrastructure</code> .
ARC-01-04	Domaine pur	<code>domain/</code> n' importe ni FastAPI, ni SQLAlchemy, ni MinIO, ni Celery, ni client LLM.
ARC-01-05	Application séquence	<code>application/</code> exécute une commande sur son root, déclenche événements/outbox et lit des références. Il n' écrit pas un autre root.
ARC-01-06	Infrastructure remplaçable	SQLAlchemy, PostgreSQL, MinIO, Docling, API Manus et workers vivent hors du domaine.
ARC-01-07	Interfaces minces	HTTP/Web/MCP convertissent l' entrée, résolvent tenant/acteur, appellent l' application et sérialisent la réponse. Aucune règle métier dans les routes.

ARC-01-08	Tests comme frontières	Les interdits d' import, de cross-write et de cascade ORM sont testés automatiquement.
ARC-01-09	Pas de dossier fourre-tout	core , common , utils , helpers , services et misc sont interdits sans décision ADR explicite.
ARC-01-10	Une structure créée à temps	Aucun module ou sous-dossier n' est créé « au cas où ».

3. Arborescence de démarrage : premier commit V8

Cette arborescence est celle qui doit être créée à l' initialisation du dépôt. Elle correspond strictement au premier slice `Case + Consultation/DceVersion + Decision` et à son socle opérationnel.

Plain Text

```
smart-ao-v8/
├── README.md
├── pyproject.toml
├── uv.lock
├── .env.example
├── .gitignore
├── Makefile
├── docker-compose.yml
├── compose.dev.yml
├── alembic.ini
├── ruff.toml
├── .github/
│   └── workflows/
│       └── ci.yml
├── docs/
│   ├── README.md
│   ├── PROJECT_STATE.md
│   ├── DECISION_LOG.md
│   └── reference/
│       ├── CHARTE_RECONSTRUCTION_SMART_AO_V8.md
│       ├── SMART_AO_V8_DOCUMENTATION_MAP.md
│       ├── SMART_AO_V8_DOMAIN_01_AGGREGATE_OWNERSHIP_MATRIX.md
│       ├── SMART_AO_V8_DOMAIN_03_STATE_MACHINES_INVARIANTS_FIRST_SLICE.md
│       └── SMART_AO_V8_APP_01_CONTRATS_PYDANTIC_PREMIER_SLICE.md
```

- └─ SMART_A0_V8_TEST_01_PLAN_TESTS_PREMIER_SLICE.md
- └─ SMART_A0_V8_DATA_01_MAPPING_PERSISTENCE_ALEMBIC_PREMIER_SLICE.md
- └─ SMART_A0_V8_ARC_01_CONTRAT_ARBORESCENCE_MODULES.md
- └─ adr/
 - └─ README.md

- └─ backend/

- └─ app/

- └─ bootstrap/
 - └─ application.py
 - └─ configuration.py
 - └─ container.py
 - └─ module_registry.py

- └─ modules/

- └─ case/
 - └─ domain/
 - └─ application/
 - └─ infrastructure/
 - └─ public/

- └─ dce/
 - └─ domain/
 - └─ application/
 - └─ infrastructure/
 - └─ public/

- └─ decision/
 - └─ domain/
 - └─ application/
 - └─ infrastructure/
 - └─ public/

- └─ platform/

- └─ persistence/
 - └─ events/
 - └─ security/
 - └─ storage/
 - └─ observability/

- └─ interfaces/

- └─ http/
 - └─ routes/
 - └─ dependencies/
 - └─ middleware/
 - └─ error_mapping.py

- └─ alembic/

- └─ env.py
 - └─ script.py.mako

```
└─ versions/
  │ 20260813_0001_platform_command_durability.py
  │ 20260813_0002_consultation_dce.py
  │ 20260813_0003_case.py
  └─ 20260813_0004_decision.py
└─ tests/
  │ conftest.py
  │ factories/
  │ domain/
  │ │ case/
  │ │ dce/
  │ │ └─ decision/
  │ application/
  │ integration/
  │ api/
  │ architecture/
  │ concurrency/
  │ security/
  └─ process/
web/
├─ package.json
├─ vite.config.ts
└─ src/
  │ app/
  │ features/
  │ │ case/
  │ │ dce/
  │ │ └─ decision/
  │ shared/
  └─ infrastructure/
reference_data/
└─ README.md
fixtures/
├─ dce/
└─ synthetic/
scripts/
├─ dev/
├─ test/
└─ verification/
ops/
├─ docker/
└─ README.md
```

3.1. Pourquoi cette structure est volontairement plus petite

Proposition cible reçue	Décision ARC-01 au premier commit	Motif
Modules nombreux : knowledge, cognitive, execution, pricing, submissions, collaboration, etc.	Non créés maintenant.	Aucun aggregate du premier slice ne leur appartient encore.
Sous-dossiers workers, agents, queue, DLQ, realtime, MCP.	Non créés maintenant.	Ce sont des mécanismes futurs ; les créer vides ferait croire qu’ils sont conçus.
Tests à la fois dans chaque module et à la racine.	<code>backend/tests/</code> unique au départ.	Une seule convention de test est plus simple ; les tests peuvent être regroupés par module plus tard si nécessaire.
Arborescence documentaire avec beaucoup de renommages.	<code>docs/reference/</code> conserve les noms existants ; <code>PROJECT_STATE.md</code> et <code>DECISION_LOG.md</code> deviennent vivants.	Préserve les liens actuels et facilite la reprise inter-session.
<code>migrations/</code> à la racine.	<code>backend/alembic/</code> unique, conformément à DATA-01.	Une seule vérité pour Alembic ; pas de double arborescence de migration.

4. Module métier : structure interne obligatoire

Lorsqu’ un module devient réel, sa structure minimale est la suivante.

Plain Text

```
backend/app/modules/<module>/
├─ domain/
│   ├── aggregates/
│   ├── entities/
│   ├── value_objects/
│   ├── events.py
│   ├── errors.py
│   └── policies.py
```

```

├── ports.py
├── application/
│   ├── commands/
│   ├── queries/
│   ├── handlers/
│   └── processes/
├── infrastructure/
│   ├── persistence/
│   └── adapters/
└── public/
    ├── commands.py
    ├── queries.py
    ├── events.py
    └── contracts.py

```

Un sous-dossier n' est créé que s' il contient au moins un élément réel. Ainsi, un module qui ne possède pas encore de `policy` ne crée pas `domain/policies/` vide ; une commande peut rester dans `application/commands.py` avant d' être éclatée si elle est seule.

Couche	Responsabilité	Dépendances autorisées	Dépendances interdites
domain	Aggregate, entités, VO, règles, événements internes, erreurs et ports abstraits.	Standard library, petits types de kernel explicitement admis.	FastAPI, Pydantic HTTP, SQLAlchemy, MinIO, Celery, LLM, réseau, modèle ORM d' un autre module.
application	Commandes, queries, handlers, Unit of Work, autorisation contextualisée, Process Managers.	Son domaine, son <code>public</code> , ports, contrats publics d' autres modules.	Internals d' un autre module, route FastAPI, client LLM direct.
infrastructure	SQLAlchemy, repository, MinIO, adaptateur API, calculateur et implémentation de ports.	Son module, <code>platform</code> , SDK technique.	Logique métier transversale, handler d' un autre module.
public	Contrats stables exportés pour les autres modules : commandes, queries, événements, types.	Types volontairement exportés, aucun ORM.	Objet riche mutable du domaine, session, repository concret.

5. Modules V8 : carte cible et déclencheur de création

L'arborescence cible reprend l'intention de la proposition reçue, mais les modules sont nommés d'après les bounded contexts réellement fixés dans DOMAIN-01.

Module cible	Contextes / aggregates	Créer lorsque...	Interdits principaux
case	AFF/Case	Créé immédiatement.	Ne possède DCE, Decision, Task, prix ou dépôt.
dce	DCE/Consultation , DceVersion , DceDocument , SourceStatement .	Créé immédiatement.	Ne produit pas Requirement, prix, Decision ou ImpactAssessment.
decision	DEC/Decision , contextes, conditions.	Créé immédiatement.	Ne modifie pas Case ni prix/dépôt.
enterprise	ORG/CompanyProfile , capacités, références, partenaires, politiques.	Premier écran bibliothèque/entreprise.	Ne devient pas un tenant/auth global.
opportunity	OPP/OpportunityProfile , Opportunity.	Début de la veille.	Ne convertit pas directement l'opportunité en Case par mutation directe.
evidence	PRF/EvidenceRecord , versions, droits d'usage.	Première bibliothèque de preuves.	Ne conclut pas seule à la conformité.
analysis	ANA/SourceAssertion , Requirement, Coverage, Assessment, Finding, Risk, ProtectionPlan.	Premier DCE analysé réellement au-delà du classement.	Ne fusionne jamais source, fait, exigence et décision.
membership	ASN/Membership , Assignment, DelegationGrant.	Premier collaborateur réel.	Ne transporte aucune donnée de prix.
work	TSK/Task , DMD/Request , REV/Review .	Premier travail préparatoire assignable.	Ne possède pas DCE, preuve, Case ou Decision.

preparation	PRE/PreparationPackage , ResponseDraft, SNP/Snapshot , TRN/Transmission .	Premier wizard collaborateur.	Ne possède aucun prix ni décision patron.
sharing	SHR/ExternalShare .	Premier partage partenaire/externe.	Ne copie pas les documents partagés.
impact	IMP/ImpactAssessment .	Premier rectificatif DCE traité.	Ne rend pas lui-même un root externe obsolète ; il émet un impact.
pricing	PRX/PricingScenario , OfficialPricingVersion.	Après socle décision/DCE ; accès patron uniquement.	LLM, UI, collaborateur, mutation de Decision/Submission.
submission	DEP/SubmissionPackage , tentative, reçu.	Après prix officiel et décision.	Ne déclare jamais un accusé sans preuve.
execution	Continuité après attribution.	Après MVP réponse AO validé.	Ne modifie jamais l' historique de réponse.
knowledge	Index, retrieval, classement de contexte, recherche interne.	Lorsque la recherche documentaire devient un besoin réel.	N' est pas propriétaire de preuve, exigence ou Decision.
cognitive	Orchestration LLM, candidats structurés, agents et tools.	Lorsque l' analyse assistée entre dans le code.	Aucune mutation directe de domaine, Decision, prix ou dépôt.

La distinction de la proposition entre **knowledge** et **cognitive** est retenue. **knowledge** récupère et assemble du contexte ; **cognitive** appelle un modèle et produit un résultat candidat structuré. Une commande métier validée reste la seule voie de mutation durable.

6. Plateforme transversale : périmètre étroit

backend/app/platform/ n' est pas un nouveau **core** fourre-tout. Il n' héberge que des mécanismes transversaux sans vocabulaire métier BTP.

Sous-module plateforme	Peut contenir	Ne peut pas contenir
------------------------	---------------	----------------------

persistence	Base SQLAlchemy commune, session factory, Unit of Work, conventions de révision.	Repository métier qui mélange les roots.
events	Domain Event envelope, outbox, inbox, déduplication, publication et retry.	Règle « une Case devient prête à chiffrer ».
security	Résolution identité/tenant, autorisation générique, audit technique.	Règle métier « un collaborateur ne voit jamais le prix » sans politique du module pricing.
storage	Port objet, checksum, adaptateur MinIO et anti-malware technique.	DceVersion, EvidenceRecord ou règle de validité métier.
observability	Logs structurés, métriques, trace/correlation id.	Un état de travail ou une décision.
runtime	Worker bootstrap, scheduler, lifecycle.	Process Manager ou workflow métier spécifique.
llm	Gateway Manus, providers, structured output, sécurité de transport.	Agent ayant le droit de changer un aggregate.

Les futures familles `realtime` , `reference_data` et `mcp` n' apparaissent qu' au moment de leur premier besoin vérifiable.

7. Interfaces : API et frontend

7.1. Backend HTTP

Plain Text

HTTP request

- validation Pydantic APP-01
- résolution serveur tenant + acteur
- autorisation d'accès
- construction de la commande application
- handler
- résultat APP-01
- sérialisation HTTP

Une route ne choisit jamais une transition métier avec un `if`. Elle ne charge pas un modèle ORM, ne crée pas d'event et ne publie pas d'outbox directement.

Chemin	Rôle
<code>backend/app/interfaces/http/routes/</code>	Regroupe les routes FastAPI par contrat public.
<code>backend/app/interfaces/http/dependencies/</code>	Construit <code>ServerResolvedContext</code> , Unit of Work, clients et dépendances.
<code>backend/app/interfaces/http/middleware/</code>	Correlation ID, sécurité HTTP, logging, traitement erreurs global.
<code>backend/app/interfaces/http/error_mapping.py</code>	Convertit les erreurs d'application en <code>CommandRejected</code> APP-01.

7.2. Frontend React

Le frontend est organisé par capacité visible, pas par tables PostgreSQL.

Plain Text

```
web/src/
├─ app/           # router, providers, bootstrapping
├─ features/      # case, dce, decision puis futurs features
├─ shared/        # UI atomique, formulaires, feedback, statuts
└─ infrastructure/ # client HTTP, auth, cache, telemetry
```

Une feature React consomme uniquement un **read model** et un **contrat de commande**. Elle ne connaît ni modèle ORM, ni règle de domaine interne, ni clé d'API LLM.

8. Règles de dépendance et d'import

8.1. Règle d'or

Plain Text

```
UN MODULE PEUT :
- utiliser ses internals ;
- importer un contrat public d'un autre module ;
- lire une référence versionnée via un port autorisé ;
- publier un événement interne / demander une commande aval corrélée ;
```

- implémenter ses propres ports d'infrastructure.

UN MODULE NE PEUT PAS :

- modifier un aggregate appartenant à un autre module ;
- importer les internals d'un autre module ;
- accéder à sa session ou ses repositories concrets ;
- partager ses modèles ORM ;
- placer une règle métier dans une projection ;
- laisser un agent ou une IA modifier directement le domaine.

8.2. Matrice d' import du premier slice

Origine	Peut importer	Ne peut pas importer
case.domain	Standard library, types de kernel admis.	dce.* , decision.* , SQLAlchemy, FastAPI.
case.application	case.domain , dce.public , decision.public , platform ports.	dce.domain , dce.infrastructure , decision.domain , modèles ORM externes.
dce.domain	Standard library, types de kernel admis.	case.* , decision.* , SQLAlchemy.
dce.application	dce.domain , case.public , platform ports.	case.domain , case.infrastructure , Decision internals.
decision.domain	Standard library, types de kernel admis.	case.* , dce.* , SQLAlchemy.
decision.application	decision.domain , case.public , dce.public , platform ports.	case.domain , dce.domain , handler/repository externe.
platform.*	Bibliothèques techniques et contrats abstraits.	Aggregates métier, règle BTP ou écran.
interfaces.http	*.public , bootstrap, interfaces de plateforme.	Domain/ORM/repository direct.
web	OpenAPI/read models/contracts frontend générés ou explicitement exportés.	Backend Python interne et base de données.

8.3. Exceptions admises

Aucune exception d' import n' est implicite. Une exception temporaire requiert : une ADR, une date d' expiration, un test d' architecture et le propriétaire chargé de la supprimer.

9. Tests d' architecture obligatoires

ARC-01 complète TEST-01. Les tests suivants existent au premier commit ; ils échouent si l' arborescence est trahie.

Test	Vérification
test_module_cannot_import_other_module_internals	Seuls les imports vers other_module.public sont permis.
test_domain_has_no_framework_or_orm_dependency	Aucun import FastAPI/SQLAlchemy/MinIO/LLM dans domain .
test_repository_handles_only_owned_root	Un repository ne sauvegarde qu' un root propriétaire.
test_handler_does_not_call_other_module_handler	Les effets inter-modules passent par event/process/commande publique.
test_no_cross_aggregate_orm_cascade	Pas de delete-orphan ou ON DELETE CASCADE cross-root.
test_interface_does_not_import_domain_or_orm	Les routes utilisent seulement contrats publics et application.
test_projection_is_read_only	Un projecteur ne peut appeler aucun repository de mutation.
test_cognitive_has_no_domain_mutation_path	Un résultat LLM ne peut pas écrire directement un aggregate.
test_pricing_contracts_are_not_importable_by_colaborator_features	Protection structurelle supplémentaire de la confidentialité financière.

Ces tests seront implémentés par import-linter , inspection AST et inspection de SQLAlchemy.metadata selon le cas. Le test ne cherche pas à deviner l' architecture : il encode cette présente règle.

10. Documentation vivante dans le dépôt

Fichier	Mise à jour	Rôle
docs/PROJECT_STATE.md	À chaque fin de session ou commit significatif.	Slice en cours, dernier commit vert, migrations, tests passants, risques ouverts et prochaine action exacte.
docs/DECISION_LOG.md	Dès qu' un arbitrage change un contrat ou une structure.	Historique concis des décisions qui ne méritent pas encore une ADR complète.
docs/adr/ADR-XXX-*.md	Décision difficile, coûteuse ou durable.	Conserver contexte, alternatives, décision et conséquences.
docs/reference/	Quand un contrat normatif V8 est modifié.	Copie versionnée et stable des documents fondateurs.
README.md	À chaque changement d' onboarding ou de commande développeur.	Démarrage rapide d' un nouveau développeur.

10.1. Format minimal de PROJECT_STATE.md

Markdown

```
# PROJECT_STATE

## Slice courant
Case + Consultation/DceVersion + Decision

## Dernier état vert
- Commit : <sha>
- Migration Alembic : <revision>
- Commande : `pytest`
- Résultat : <nombre> passed

## Ce qui est terminé
- <items vérifiés>

## Prochaine action unique
- <fichier/test/commande à réaliser>

## Décisions ouvertes
- <question, owner, échéance>
```

Risques bloquants

- <aucun ou description>

11. Plan de création incrémental du dépôt

Étape	Ce qui est créé	Interdit à cette étape	Sortie vérifiable
1	Racine, docs, backend/web minimaux, CI, Docker Compose, PROJECT_STATE.md .	Modules métier futurs et workers vides.	Clone, installation, lint et smoke test.
2	Platform persistence/events/security minimal, Alembic, migrations DATA-01.	Queue distribuée, DLQ, realtime, LLM, MCP.	alembic upgrade head sur PostgreSQL propre.
3	Module case en domaine pur + tests rouges/verts.	ORM dans domaine, endpoint avant règles.	Tests CASE-* applicables verts.
4	Module dce en domaine pur/persistence + tests.	OCR/Docling/IA de production.	Tests CONS-*, DCE-*, corpus immuable.
5	Module decision + Process Manager minimal + tests.	Prix, tâches, UX complète.	Tests DEC-*, concurrence Go/No-Go et outbox verts.
6	Interfaces HTTP APP-01 + premières features React.	Cockpit complet et navigation large.	API/RYOW/idempotence verts, démo du scénario noyau.
7	Slices suivants, un à un, avec contrat de domaine/état avant code.	Gros refactor transversal non justifié.	Chaque slice ajoute son propre test et ADR si nécessaire.

12. Arborescence cible à maturité

Cette section n' est pas un ordre de création. Elle confirme simplement où les futurs slices se rangeront lorsqu' ils auront leurs contrats et tests.

Plain Text

```
backend/app/modules/  
├─ case/  
├─ dce/  
├─ decision/  
├─ enterprise/  
├─ opportunity/  
├─ evidence/  
├─ analysis/  
├─ membership/  
├─ work/  
├─ preparation/  
├─ sharing/  
├─ impact/  
├─ pricing/  
├─ submission/  
├─ execution/  
├─ knowledge/  
└─ cognitive/
```

La proposition initiale séparait `tasks` et `collaboration`. ARC-01 conserve cette idée sous les noms `work` et `membership/preparation` : le work item générique ne doit pas devenir propriétaire de l' affectation, de la transmission, de la preuve ou de la préparation. Si l' expérience montre que cette découpe complique inutilement le code, une ADR pourra décider de créer un module `collaboration` façade qui dépend seulement des contrats publics concernés, sans fusionner les roots.

13. Décisions de gel ARC-01

1. Le dépôt V8 commence avec `case`, `dce` et `decision`, et non avec l' ensemble des modules futurs.
2. Chaque module réel respecte les couches `domain`, `application`, `infrastructure` et `public` ; les dossiers internes n' apparaissent qu' en présence de code réel.
3. Les contrats actuels sont importés dans `docs/reference/` sans renommage destructif ; `PROJECT_STATE.md` est le point de reprise rapide.
4. La couche `platform` reste transversale et sans logique métier BTP ; elle ne devient pas un fourre-tout.

5. Les routes FastAPI n' importent aucun aggregate de domaine ou modèle ORM et n' exécutent aucune logique métier.
 6. Les modules ne communiquent qu' au travers de contrats publics, références versionnées, événements/outbox et commandes aval corrélées.
 7. Les tests d' architecture sont bloquants dès le premier commit ; une dépendance interdite est une régression fonctionnelle.
 8. Les futurs modules `knowledge` , `cognitive` , `pricing` , `submission` , `execution` et collaboration ne sont créés qu' au début de leur slice, avec contrat et tests préalables.
 9. Aucune nouvelle classe, table, colonne ou dossier ne peut être ajouté « au cas où » sans être relié à une commande, un invariant, un test ou une décision ADR.
-

Références internes

- `CHARTE_RECONSTRUCTION_SMART_AO_V8.md` .
 - `SMART_AO_V8_DOCUMENTATION_MAP.md` .
 - `SMART_AO_V8_DOMAIN_01_AGGREGATE_OWNERSHIP_MATRIX.md` — DOMAIN-01 v1.1.
 - `SMART_AO_V8_DOMAIN_03_STATE_MACHINES_INVARIANTS_FIRST_SLICE.md` — DOMAIN-03 v1.0.
 - `SMART_AO_V8_APP_01_CONTRATS_PYDANTIC_PREMIER_SLICE.md` — APP-01 v1.0.
 - `SMART_AO_V8_TEST_01_PLAN_TESTS_PREMIER_SLICE.md` — TEST-01 v1.0.
 - `SMART_AO_V8_DATA_01_MAPPING_PERSISTENCE_ALEMBIC_PREMIER_SLICE.md` — DATA-01 v1.0.
-

Fin de ARC-01 — Contrat d' arborescence, modules et dépendances — version 1.0